Verifiable Credential Wallet PWA

Frontend Developer Assignment — UIDAI Innovation Vertical

> **Candidate:** Manoj
> **Repository:** To be added before submission
> **Stack:** React, TypeScript, Redux Toolkit, Vite, Vitest, React Testing Library

Build passed Lint passed 17 tests passed

This submission implements a responsive sandbox credential wallet using mock identity data. It explicitly covers dashboard state management, secure reveal behavior, accessibility, offline credential retrieval, and automated security-timeout testing.

Part 1 — Dashboard and State Management

Implementation

- A responsive credential grid presents mock Aadhaar and eKYC credentials.
- Redux Toolkit stores credentials, request status, validation failures, and the last synchronization time.
- The application requests **GET /api/credentials** and validates every returned object at runtime before storing it.
- Dedicated loading, error, empty, and populated views ensure predictable behavior for every request state.
- The error view presents a retry action and a useful failure message.

```
Request lifecycle: idle → loading → succeeded | failed
Success with []: empty-state panel
Success with data: responsive credential grid
Failure: alert panel with retry button
```

Part 2 — SecureDataMask and Accessibility

Secure reveal lifecycle

- The reusable component receives a sensitive string but renders a masked form by default.
- Activating “Tap to reveal” copies the value into temporary local component state.
- A timer clears that state and restores the mask after 10 seconds.
- “Hide now” immediately clears the revealed state, and unmounting clears any pending timer.
- Numeric 12-digit values use Aadhaar-style grouping; opaque alphanumeric identifiers remain unmodified while visible.

Accessibility

- A native button provides keyboard operation and standard focus behavior.
- **aria-controls** connects the control to the displayed value.
- **aria-pressed** communicates reveal state.
- A polite live region announces reveal and hide changes.
- The dashboard includes semantic landmarks, a skip link, visible focus styles, reduced-motion support, and loading/error announcements.

Part 3 — Offline Support and Automated Testing

Service worker strategy

- The production application registers a vanilla service worker.
- **GET /api/credentials** uses a network-first strategy.
- Successful responses are cloned and fully written to Cache Storage before being returned.
- Network exceptions and unsuccessful HTTP responses fall back to the last cached credential list.
- If no cached response exists, the service worker returns an empty JSON list marked with an offline-fallback header.
- Same-origin application resources are cached as they are requested, with navigation fallback to the cached application shell.

Automated coverage

- Default masking and absence of the full sensitive value
- Reveal followed by automatic masking after exactly 10 seconds
- Immediate manual hiding
- Keyboard activation and live-region announcement
- Timer cleanup when the component unmounts
- Dashboard loading, success, empty, HTTP failure, and malformed-payload states
- Exact API endpoint usage
- Service-worker network success, HTTP failure cache fallback, and no-cache offline fallback

```
npm test       3 test files, 17 tests passed
npm run lint   0 warnings, 0 errors
npm run build  TypeScript and Vite production build passed
```

Security and Scope

> **Sandbox only:** All credentials are mock data. Cache Storage demonstrates the assignment's offline requirement and is not encrypted storage. Real identity data must not be entered.

A production wallet would additionally require an approved threat model, strong authentication and authorization, encrypted local storage with secure key management, CSP and server-side security headers, dependency monitoring, and independent security and accessibility audits.

Authentication, credential issuance, and secondary navigation are represented as sandbox UI and remain outside the three requested coding prompts.

Run and Verify

```
npm install
npm run dev
```

```
npm test
npm run lint
npm run build
```